VAST Builders Corner: Build Day Guide

The infrastructure is already running, so you skip straight to the interesting part: turning hours of video into something that searches, reasons, and acts.

1. Kickoff

What you're building

An app that understands video and does something with it. Search hours of footage in plain language, ask what happened, detect and track objects, or call an action when something matters.

Pick one idea and build a working app.

2. Skills, Skills, Skills

The core skills are `ingest/` and `retrieval/`, in `.cursor/skills/`. More skills cover the rest of the deployed stack, if you want to explore. Each skill guides the coding agent: the endpoint, the request, the response, and what to do when it fails.

Describe what you want and the coding agent loads the matching skill.

```
"re-ingest the warehouse video with a prompt about safety gear"
"find people near the entrance after 6pm"
"summarize what happens in the warehouse video"
```

Skills read what they need from environment variables, so nothing should ask you for a password or a URL. If something isn't working, ask for help.

Re-ingest: running it again

> ⚠️ **IMPORTANT:** Avoid re-ingesting large chunks of video (i.e. hours of video). Start with a few clips and read the captions first.

The video is already indexed. Re-ingest means running it through the pipeline again with a different prompt, so the descriptions match what you're building.

Skill	Use it to
`reingest-videos`	Re-run a whole indexed video with a new prompt or metadata
`reingest-chunk`	Re-run one specific chunk, found by filename, scene, date, or camera

Re-ingesting takes a few minutes. Every segment is described, embedded, and detected again before it becomes searchable. Ask Cursor to confirm before you go looking.

Find what's missing

Search for something your idea needs that the existing descriptions probably don't mention. Counts of people. What someone is carrying. Whether a vehicle stopped.

If it comes back empty, that's not a broken search. It means the ingestion prompt never asked about it, so nothing was written down.

Re-ingest with your prompt

```
re-ingest <that video> with a prompt that describes <what your app needs>
```

Cursor loads `reingest-videos`, shows you what it's about to re-run, and asks for the prompt. Give it a few minutes, then run the same search again. This time it matches.

> 💡 That gap, between what you searched for and what the prompt asked about, is the thing to keep in mind. Everything you can build depends on what the descriptions say.

Retrieval: Searching Video

Skill	Use it to
`search`	Find moments matching a description, with filters for time, location, camera, tags.
`agent-qa`	Ask a question and get an answer with evidence, instead of a list of hits.
`videos`	Browse what's indexed, play a clip, read its captions and detections, summarize a whole video.
`dashboard`	Check what's indexed and whether ingest is healthy.
`suggest-prompts`	Get generated example queries and notable recent events.

3. Build

What "done" looks like

You have a working index and you know how to query it. What comes next is what you build on top. A small app, agent, or a dashboard. A clear use case.

What video you have

The videos come from a few kinds of real-world footage:

- Formula racing footage (F1 Academy and Formula E)
- Bundesliga football footage
- Dashcam driving footage of vehicles and pedestrians interacting at intersections and crossings
- Overhead, multi-camera highway footage tracking vehicle movement along a stretch of interstate
- A private neighborhood camera capturing car movement

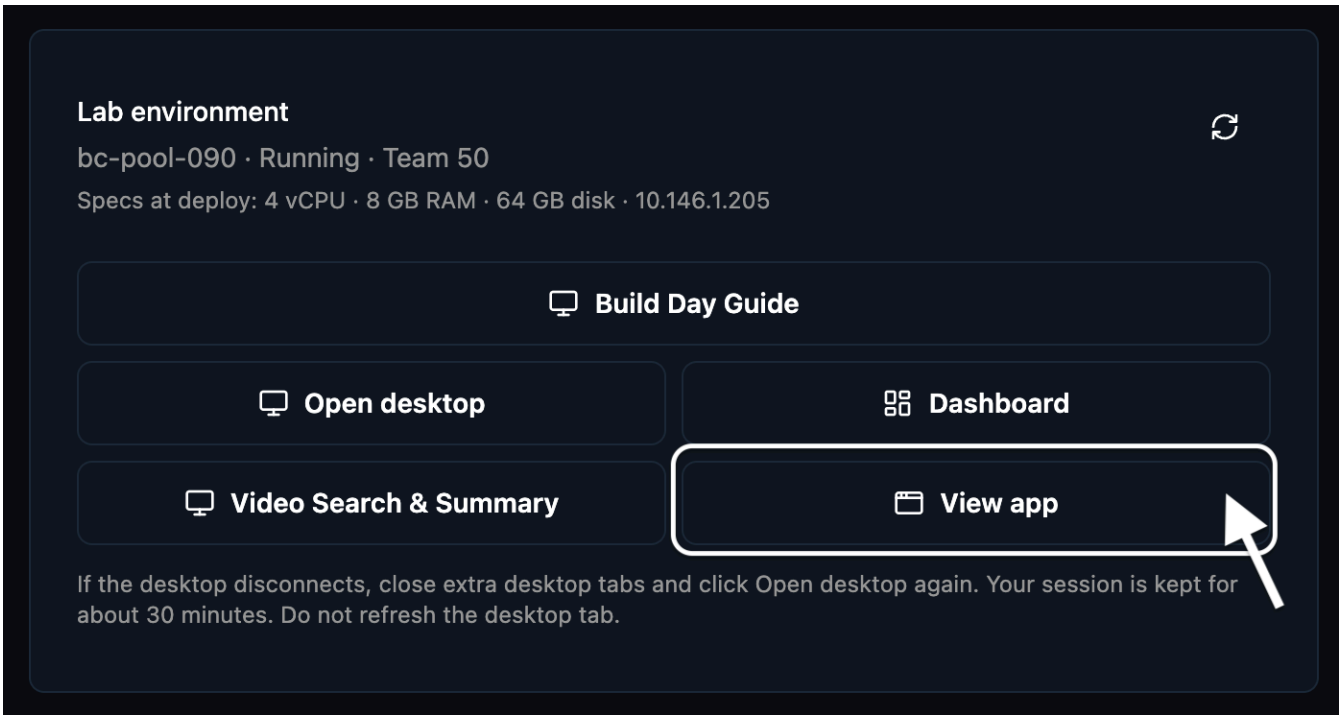
Explore what's actually indexed in the Video Search & Summary UI.

Your index is already full and searchable. For the full list of folders, camera IDs, use cases, and worked example queries, see the Architecture Reference's Video corpus section.

The loop

1. **Pick a use case**
2. **Build an app or agent**
3. **Deploy and iterate**

> 💡 Once you've deployed with `/deploy-app-no-registry`, check out your app in your own browser at workshop.thecosmoslabs.com — click **App**.



If the existing captions cover what you need, you never have to think about prompts. If they don't, re-ingest the footage with a different prompt.

> 💡 **Start with a few clips.** Read the captions that come back before you re-ingest anything at volume.

LLM access

Search and Q&A come from your VSS instance. Anything your app decides on top of that, classifying results, drafting a summary, choosing an action, runs on serverless LLM inference from Weights & Biases.

Your `WANDB_` keys are already in your environment. Point your own app's reasoning or agent at the inference endpoint using those. If you need more credits, reach out to the Weights & Biases by CoreWeave team :)

The skills used by Cursor can be used by agent frameworks too. They follow the standard `SKILL.md` format, so most frameworks load them straight from `.cursor/skills/`.

Reference

Health check

If something isn't working, pull first. The repo is pre-cloned and may be behind. Type the following into Cursor:

```
run a git pull
```

Then ask Cursor:

```
check that everything is working
```

If it fails, or you need more help, run `/ask-cosmos`. The skill writes a note with the relevant details but **does not post it**. Copy it and share it with an organizer, and we'll debug it together.

Architecture Reference

For the full video corpus, metadata reference, and example queries, see the Architecture Reference.

More prompts

For more prompts on the formula racing and football tracks, see `prompts/`.